Documento Técnico del Esquema de Integración Eulalia V2

Idioma: Español
Estado: Prueba de concepto técnica (PoC)
Alcance: Documentación de integración; no constituye preparación productiva ni certificación electoral

1. Propósito y Alcance

Este documento actualiza el esquema de integración de Eulalia para incluir el servicio independiente `eulalia-backend-cardano`. Describe cómo se relacionan la aplicación móvil, el frontend web, el backend .NET, la infraestructura Identus, PostgreSQL y la PoC de transacciones en Cardano Preview.

El alcance se limita al código, READMEs y evidencias disponibles en los repositorios de Eulalia. No afirma un despliegue productivo, certificación electoral, aprobación de custodia ni una transacción Cardano Preview E2E exitosa.

2. Cambio de Versión

La V2 incorpora el backend Cardano Preview como componente TypeScript/Fastify separado. Acepta compromisos de privacidad para eventos de afiliación y cancelación, construye metadata acotada, firma con una wallet institucional de demostración mantenida en el entorno del servidor, envía mediante Blockfrost y guarda el estado de demostración en un archivo JSON local.

3. Mapa de Componentes

```
Ciudadano
  |
  v
eulalia-app (Flutter) -------\
                              > eulalia-backend (.NET 8) ---> PostgreSQL
eulalia-frontend (React) ----/             |
                                           +--> Cloud Agent Identus ---> soporte Prism/Cardano

Evento de afiliación o cancelación
  |
  v
eulalia-backend-cardano (TypeScript/Fastify) ---> Blockfrost ---> Cardano Preview
              |
              +--> estado de transacciones en JSON local
```

Componente	Tecnología	Responsabilidad	Interfaz o dependencia
`eulalia-app`	Flutter/Dart	Onboarding ciudadano, interacción con wallet/SSI, biometría y UI de afiliación	Llama a la API .NET; URL base predeterminada `http://localhost:5219`
`eulalia-frontend`	React, Vite, TypeScript	Registro web, autenticación, organizaciones, afiliaciones y estados	Llama a la API .NET mediante `VITE_API_BASE_URL`
`eulalia-backend`	.NET 8 ASP.NET Core/C#	API REST, JWT, reglas de ciudadanos y afiliaciones, orquestación SSI y webhooks	PostgreSQL y Cloud Agent Identus; Swagger en la URL local documentada

Componente	Tecnología	Responsabilidad	Interfaz o dependencia
Identus/Cloud Agent	Hyperledger Identus 1.39, Docker, Prism, Vault	Conexiones DIDComm, ofertas de credenciales, persistencia del agente e infraestructura SSI	Cloud Agent mediante `/cloud-agent`; la instancia PostgreSQL Docker soporta el stack
PostgreSQL	PostgreSQL 13+	Datos persistentes de la aplicación y del agente en la arquitectura existente	Base local de la aplicación y base Docker de Identus documentadas por separado
`eulalia-backend-cardano`	Node.js 20+, TypeScript, Fastify, Lucid Evolution	Valida y registra compromisos de eventos de afiliación/cancelación en Preview	API HTTP en `127.0.0.1:3000`; proveedor Blockfrost Preview
Cardano Preview/Blockfrost	Cardano Preview y API Blockfrost	Acceso al proveedor/red, envío y consulta de estado	`https://cardano-preview.blockfrost.io/api/v0` por defecto

Repositorios: móvil, frontend, backend .NET, Identus y backend Cardano.

4. Flujo SSI e Identidad

1. El ciudadano se registra y autentica desde la app móvil o el frontend mediante la API .NET.
2. El backend .NET crea una invitación DIDComm mediante el Cloud Agent Identus.
3. La app presenta la URL de invitación como código QR para una wallet.
4. La wallet acepta la conexión; Identus envía un evento webhook al backend .NET.
5. El backend crea una oferta de credencial, guarda los identificadores del holder y la credencial, y expone el estado SSI consolidado.
6. La afiliación se permite únicamente después de cumplir las validaciones documentadas de credencial y biometría.
7. El servicio Cardano separado recibe un evento de afiliación o cancelación con datos de privacidad cuando corresponde registrarlo en Preview.

El flujo de identidad SSI y el registro de eventos Cardano son límites separados: la metadata Cardano no es el almacén de credenciales SSI ni reemplaza la base de datos de la aplicación.

5. Flujo Cardano de Afiliación y Cancelación

El servicio Cardano expone estas operaciones autenticadas:

Método	Endpoint	Propósito
GET	`/health`	Salud pública del servicio e indicación de red `Preview`
POST	`/api/transactions/affiliations`	Registrar un evento de afiliación
POST	`/api/transactions/cancellations`	Registrar un evento de cancelación
GET	`/api/transactions/:eventId`	Consultar el estado almacenado
POST	`/api/transactions/:eventId/refresh`	Consultar el estado actual en Blockfrost y guardarlo

Los endpoints autenticados requieren `Authorization: Bearer <INTERNAL_API_TOKEN>`. La entrada se limita a `eventId`, `affiliationId`, `organizationId` y `recordHash`. Los identificadores son acotados, se rechazan campos desconocidos y `recordHash` debe ser un commitment SHA-256 hexadecimal de 64 caracteres.

El servicio guarda `pending` antes del envío. Construye metadata acotada, firma y envía mediante Lucid y Blockfrost, y luego guarda `submitted` con `txHash`. Los errores del proveedor guardan `failed` con el código seguro

`BLOCKCHAIN_SUBMISSION_FAILED`. El modelo de estados también contempla `confirmed` y `not_found` para consultas.

Las solicitudes repetidas son idempotentes por `eventId`: se devuelve el registro existente y no se envía una segunda transacción. Las solicitudes concurrentes del mismo evento comparten la operación en curso. El endpoint de refresh actualiza un registro enviado a partir de Blockfrost; los estados terminales `confirmed` y `failed` no se refrescan.

6. Límite de Privacidad

El límite Cardano acepta únicamente commitments y metadata de evento limitada. Los campos documentados son `eventType`, `eventId`, `affiliationId`, `organizationId`, `recordHash`, `schemaVersion` y `timestamp`, con un límite de seguridad de 1.600 bytes y una etiqueta de metadata configurable.

No deben llegar on-chain la cédula, nombres, datos biométricos, registros ciudadanos sin transformar, contenido de credenciales ni otra PII. Un commitment SHA-256 permite comparar integridad posteriormente solo si el registro privado correspondiente permanece bajo control del almacenamiento de la aplicación. La clave privada CNE se configura en el servidor y no debe retornarse ni registrarse en logs.

7. Configuración y Ejecución

Stack de identidad existente

Siga los README de eulalia-identus, eulalia-backend, eulalia-frontend y eulalia-app. Los servicios locales documentados incluyen la API .NET, PostgreSQL, Cloud Agent Identus y los clientes web/móvil.

PoC Cardano

El README de Cardano documenta Node.js 20+, un proyecto Blockfrost Preview, una wallet Preview con fondos y este flujo sin secretos:

```
npm install
npm run typecheck
npm test
npm run build
npm start
```

Configure `.env` a partir de `.env.example` sin versionar secretos. Los valores requeridos son `BLOCKFROST_PROJECT_ID`, `CNE_ADDRESS`, `CNE_PRIVATE_KEY` e `INTERNAL_API_TOKEN`; los opcionales incluyen `PORT`, `HOST`, `CARDANO_NETWORK` (debe ser `Preview`), `METADATA_LABEL`, `METADATA_SCHEMA_VERSION` y `DATA_FILE`. Nunca copie valores secretos en este documento.

8. Evidencia y Limitaciones

- Los PDFs actuales `Test Report` en inglés y español de `final-report/` son la fuente de línea base y reconciliación de métricas.
- El agregado histórico de 98.0% no es un resultado actual aprobado.
- La evidencia existente muestra cobertura backend baja en el reporte referido: aproximadamente 12.4% de líneas y 25% de ramas.
- Los fallos históricos de backend y frontend, o los resultados dependientes del entorno, pueden no reproducirse en el checkout actual; siguen siendo contexto de evidencia, no una nueva afirmación de ejecución.
- No se afirma éxito E2E de una transacción Cardano Preview. Todavía se requieren credenciales Preview válidas, una wallet con fondos, acceso de red y una verificación controlada.

- La persistencia JSON es de demostración, no un almacenamiento transaccional durable.
- Una clave única de demostración en el servidor no es un diseño de custodia institucional.
- El servicio actual solo soporta Preview y no constituye un límite de producción ni de certificación electoral.

9. Recomendaciones para Piloto o Producción

1. Definir requisitos legales, de gobernanza y electorales antes de considerar autoritativo cualquier evento del ledger.
2. Sustituir la clave de demostración por custodia con Vault/HSM, rotación, controles de acceso y preferentemente aprobación institucional multifirma.
3. Sustituir JSON por almacenamiento transaccional durable, migraciones, respaldos y procedimientos de reconciliación.
4. Agregar identidad de servicio a servicio, autorización, límites de tasa, reintentos, observabilidad, alertas y auditoría.
5. Analizar correlación de commitments, filtración de metadata, replay, duplicados, autenticidad de webhooks y compromiso de claves.
6. Ejecutar una matriz reproducible en Preview y después un piloto gobernado antes de considerar Mainnet o producción.
7. Aumentar cobertura unitaria, de integración, fallos, privacidad y operación; publicar métricas con comandos y detalles del entorno.

10. Referencias Públicas

- Documentación de Hyperledger Identus
- Descripción de las redes de prueba Cardano
- Portal para desarrolladores de Cardano
- Documentación de Blockfrost
- Documentación de Fastify
- Repositorio de Lucid Evolution
- Documentación de .NET